

Optimization: Gradient Descent

Numerical Methods for Training AI Models

Firuz Kamalov

MAI 600: Mathematics and Statistics for AI

October 3, 2025

- 1 Methods of Optimization
- 2 1D Example: A Simple Parabola
- 3 2D Example: A Paraboloid
- 4 Application: Linear Regression
- 5 Application: Simple Neural Network

1. Analytic Optimization

- Uses calculus to find exact solutions.
- Involves taking the derivative (or gradient) of the function and setting it to zero.
- Solves the resulting equation(s) to find critical points (minima, maxima, or saddle points).
- **Advantage:** Provides an exact, closed-form solution.
- **Disadvantage:** Only feasible for simpler, well-behaved functions.

Two Ways to Optimize

1. Analytic Optimization

- Uses calculus to find exact solutions.
- Involves taking the derivative (or gradient) of the function and setting it to zero.
- Solves the resulting equation(s) to find critical points (minima, maxima, or saddle points).
- **Advantage:** Provides an exact, closed-form solution.
- **Disadvantage:** Only feasible for simpler, well-behaved functions.

2. Numeric Optimization

- Uses iterative algorithms to find approximate solutions.
- Starts with an initial guess and repeatedly refines it to get closer to a minimum.
- Gradient Descent is a key example.
- **Advantage:** Can be applied to very complex functions where analytic solutions are intractable.
- **Disadvantage:** Solution is an approximation; performance depends on algorithm parameters.

1D Example - Analytic Solution

Consider the function $f(x) = x^2 - 4x$.

Solving Analytically:

- 1 Find the derivative of $f(x)$:

$$f'(x) = 2x - 4$$

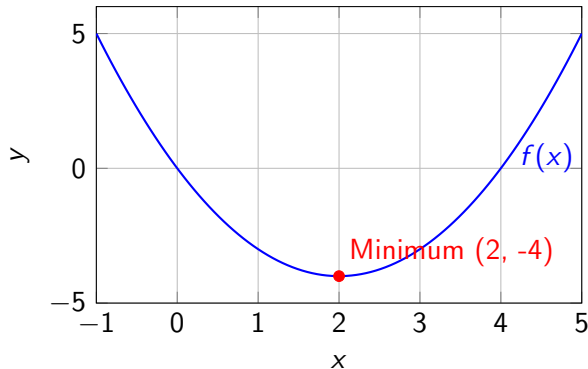
- 2 Set the derivative to zero to find critical points:

$$2x - 4 = 0$$

- 3 Solve for x :

$$2x = 4 \implies x = 2$$

The minimum is at $x = 2$, where
 $f(2) = 2^2 - 4(2) = -4$.



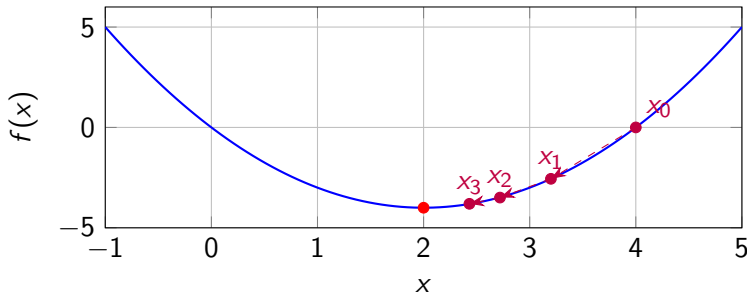
1D Example - Numeric (Gradient Descent)

Now, let's solve $f(x) = x^2 - 4x$ numerically.

- **Update Rule:** $x_{i+1} = x_i - \eta f'(x_i)$, where $f'(x) = 2x - 4$.
- **Parameters:** Let's start at a random point $x_0 = 4$ with a learning rate $\eta = 0.2$.

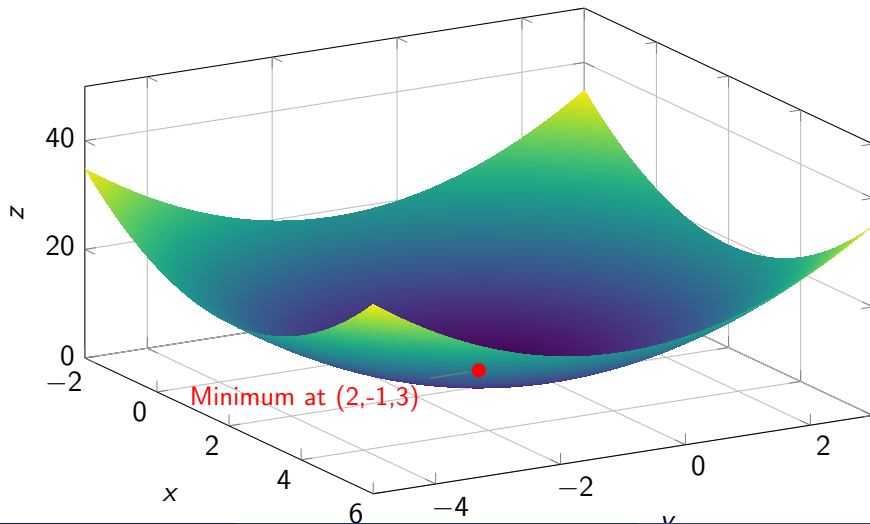
Iterations:

- $i = 0$: $x_1 = x_0 - 0.2 \cdot (2x_0 - 4) = 3.2$
- $i = 1$: $x_2 = x_1 - 0.2 \cdot (2x_1 - 4) = 2.72$
- $i = 2$: $x_3 = x_2 - 0.2 \cdot (2x_2 - 4) = 2.432$



2D Example: $f(x, y) = x^2 + y^2 - 4x + 2y + 8$

A Convex Paraboloid



2D Example - Analytic Solution

For $f(x, y) = x^2 + y^2 - 4x + 2y + 8$.

Solving Analytically:

- 1 Find the gradient, $\nabla f = \left[\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y} \right]$:

$$\frac{\partial f}{\partial x} = 2x - 4$$

$$\frac{\partial f}{\partial y} = 2y + 2$$

So, $\nabla f(x, y) = [2x - 4, 2y + 2]$.

- 2 Set the gradient to the zero vector, $\nabla f = [0, 0]$:

$$2x - 4 = 0 \implies x = 2$$

$$2y + 2 = 0 \implies y = -1$$

The minimum is at the point $(2, -1)$, where $f(2, -1) = 3$.

2D Example - Numeric (Gradient Descent)

For $f(x, y) = x^2 + y^2 - 4x + 2y + 8$.

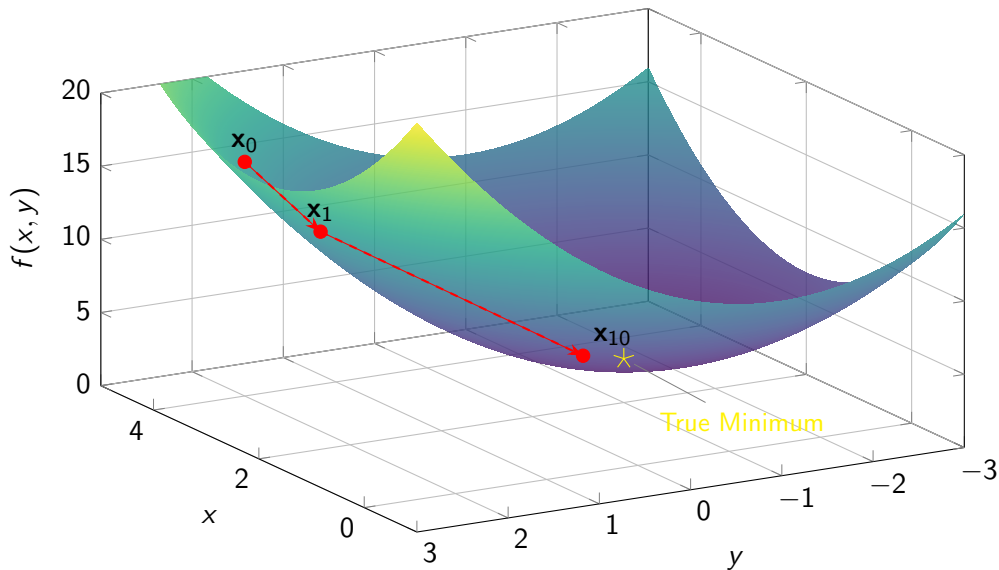
- **Update Rule (vector form):** $\mathbf{x}_{i+1} = \mathbf{x}_i - \eta \nabla f(\mathbf{x}_i)$, where $\mathbf{x} = [x, y]$.
- **Gradient:** $\nabla f(x, y) = [2x - 4, 2y + 2]$.
- **Parameters:** Let's start at $\mathbf{x}_0 = [4, 2]$ with a learning rate $\eta = 0.1$.

Iterations:

- $i = 0$:
 - $\nabla f(\mathbf{x}_0) = [4, 6]$.
 - $\mathbf{x}_1 = \mathbf{x}_0 - \eta \nabla f(\mathbf{x}_0) = [4, 2] - 0.1 \cdot [4, 6] = [3.6, 1.4]$.
- $i = 1$:
 - $\nabla f(\mathbf{x}_1) = [3.2, 4.8]$.
 - $\mathbf{x}_2 = \mathbf{x}_1 - \eta \nabla f(\mathbf{x}_1) = [3.6, 1.4] - 0.1 \cdot [3.2, 4.8] = [3.28, 0.92]$.
- $i = 10$: $\mathbf{x}_{10} = [2.215, -0.678]$

The points are iteratively approaching the minimum at $(2, -1)$.

2D Example: Gradient Descent Visualization



When Analytic Solutions are Tricky

Consider a function involving a sigmoid, common in neural networks:

$$f(x, y) = \text{sigmoid}(4x - 2y) = \frac{1}{1 + e^{-(4x-2y)}}$$

Analytic Approach:

- The gradient is non-linear and more complex:

$$\nabla f = \left[\frac{4e^{-(4x-2y)}}{(1 + e^{-(4x-2y)})^2}, \frac{-2e^{-(4x-2y)}}{(1 + e^{-(4x-2y)})^2} \right]$$

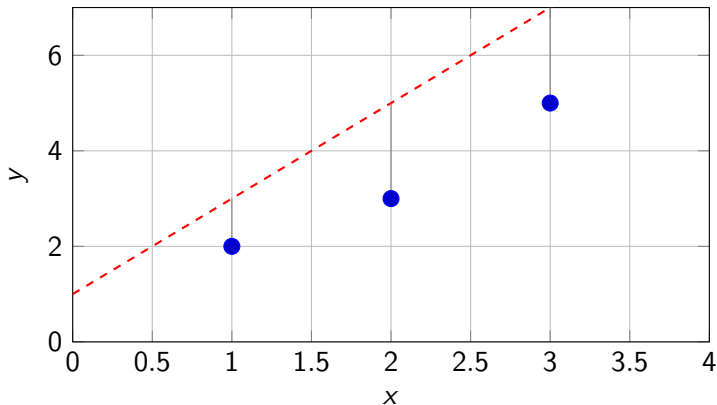
- Setting this gradient to $[0, 0]$ does not lead to a simple algebraic solution. The exponential terms make it intractable to solve for x and y directly.

Conclusion

For many real-world functions in AI, especially in deep learning, analytic solutions are impossible or impractical to find. We **must** rely on numerical methods like Gradient Descent.

Application - Linear Regression

Problem: Find the best-fit line for the points $(1, 2)$, $(2, 3)$, and $(3, 5)$.
Let's consider an initial guess for the line: $y = 2x + 1$.



How good is our initial line $y = 2x + 1$? We measure the error using the **Sum of Squared Errors (SSE)**.

$$\text{SSE} = \sum_{i=1}^n (y_i - \hat{y}_i)^2, \text{ where } \hat{y}_i = mx_i + b$$

- Point (1, 2): Error = $(y_1 - \hat{y}_1)^2 = (2 - 3)^2 = 1$.
- Point (2, 3): Error = $(y_2 - \hat{y}_2)^2 = (3 - 5)^2 = 4$.
- Point (3, 5): Error = $(y_3 - \hat{y}_3)^2 = (5 - 7)^2 = 4$.

$$\text{Total SSE} = 1 + 4 + 4 = 9.$$

Our goal is to find the values of m and b that **minimize** this SSE value.

The Gradient Vector for the Loss Function

The loss function is the Sum of Squared Errors (SSE):

$$J(m, b) = \sum_{i=1}^3 (y_i - (mx_i + b))^2$$

1. General partial derivatives:

$$\frac{\partial J}{\partial m} = -2 \sum x_i (y_i - mx_i - b)$$

$$\frac{\partial J}{\partial b} = -2 \sum (y_i - mx_i - b)$$

2. For our points (1, 2), (2, 3), (3, 5):

$$\frac{\partial J}{\partial m} = -2 \left(1(2 - m - b) + 2(3 - 2m - b) + 3(5 - 3m - b) \right) = 28m + 12b - 46$$

$$\frac{\partial J}{\partial b} = -2 \left((2 - m - b) + (3 - 2m - b) + (5 - 3m - b) \right) = 12m + 6b - 20$$

So, $\nabla J(m, b) = [28m + 12b - 46, \quad 12m + 6b - 20]$.

We find the minimum by setting the gradient vector to zero, $\nabla J(m, b) = [0, 0]$:

$$28m + 12b - 46 = 0$$

$$12m + 6b - 20 = 0$$

Equivalently:

$$14m + 6b = 23$$

$$6m + 3b = 10$$

Solving this 2×2 system gives:

$$m = 1.5, \quad b = \frac{1}{3}$$

The best-fit line is $y = 1.5x + 1/3$.

Solving Numerically with Gradient Descent

For our loss function $J(m, b)$:

- **Update Rule:** $\begin{bmatrix} m_{i+1} \\ b_{i+1} \end{bmatrix} = \begin{bmatrix} m_i \\ b_i \end{bmatrix} - \eta \nabla J(m_i, b_i)$
- **Gradient:** $\nabla J(m, b) = [28m + 12b - 46, \quad 12m + 6b - 20]$
- **Parameters:** Start at $(m_0, b_0) = (2, 1)$ with learning rate $\eta = 0.01$.

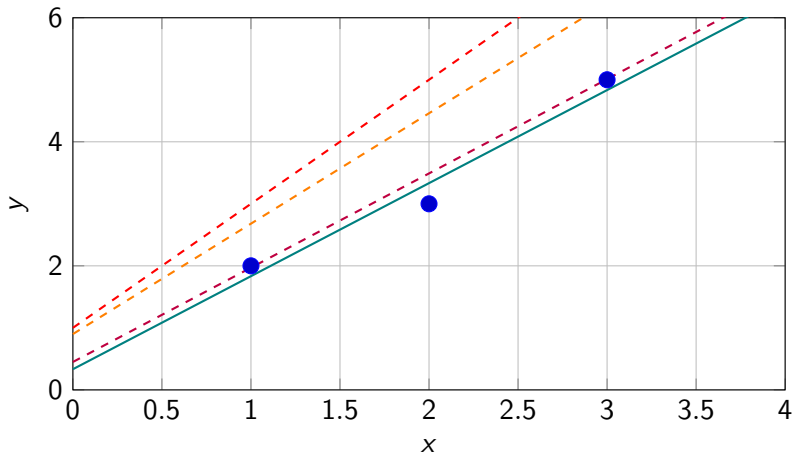
Iterations:

- **i=0:**
 - $\nabla J(2, 1) = [28(2) + 12(1) - 46, \quad 12(2) + 6(1) - 20] = [22, 10]$
 - $\begin{bmatrix} m_1 \\ b_1 \end{bmatrix} = \begin{bmatrix} 2 \\ 1 \end{bmatrix} - 0.01 \begin{bmatrix} 22 \\ 10 \end{bmatrix} = \begin{bmatrix} 2 - 0.22 \\ 1 - 0.1 \end{bmatrix} = \begin{bmatrix} \mathbf{1.78} \\ \mathbf{0.9} \end{bmatrix}$
- **i=1:**
 - $\nabla J(1.78, 0.9) = [28(1.78) + 12(0.9) - 46, \quad 12(1.78) + 6(0.9) - 20] = [14.64, 6.76]$
 - $\begin{bmatrix} m_2 \\ b_2 \end{bmatrix} = \begin{bmatrix} 1.78 \\ 0.9 \end{bmatrix} - 0.01 \begin{bmatrix} 14.64 \\ 6.76 \end{bmatrix} = \begin{bmatrix} \mathbf{1.6336} \\ \mathbf{0.8324} \end{bmatrix}$
- **i=10:** The parameters continue to approach the analytic solution.

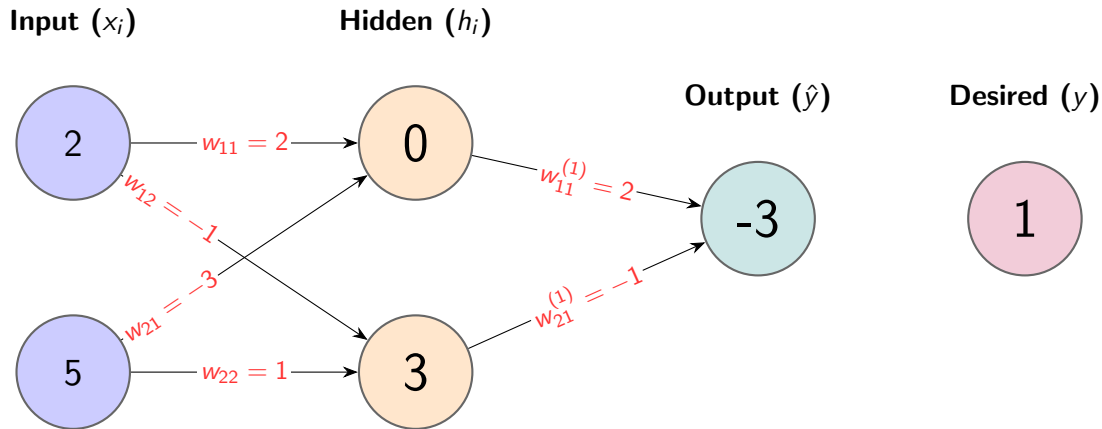
$$[m_{10}, b_{10}] \approx [1.517, 0.452]$$

Visualizing the Solution

The analytic method gives the exact optimal line $y = 1.5x + 1/3$ (solid). Gradient Descent starts with $y = 2x + 1$ (dashed red) and iteratively moves towards the optimal line.



Neural Network with ReLU Activation



① Hidden layer nodes:

- $h_1 = \text{ReLU}(x_1 w_{11} + x_2 w_{21})$
- $h_2 = \text{ReLU}(x_1 w_{12} + x_2 w_{22})$

② Output layer node: $\hat{y} = h_1 w_{11}^{(1)} + h_2 w_{21}^{(1)}$

Backpropagation: Output to Hidden Layer

Loss Function: $L = \frac{1}{2}(y - \hat{y})^2$

Initial Error Gradient:

$$\frac{\partial L}{\partial \hat{y}} = \hat{y} - y = -3 - 1 = -4$$

Gradients for each weight: $\frac{\partial L}{\partial w^{(1)}} = \frac{\partial L}{\partial \hat{y}} \times \frac{\partial \hat{y}}{\partial w^{(1)}}$

- $\frac{\partial L}{\partial w_{11}^{(1)}} = \frac{\partial L}{\partial \hat{y}} \times h_1 = (-4) \times 0 = 0$
- $\frac{\partial L}{\partial w_{12}^{(1)}} = \frac{\partial L}{\partial \hat{y}} \times h_2 = (-4) \times 3 = -12$

Update rule: $w_{new} = w_{old} - \eta \cdot \frac{\partial L}{\partial w}$ ($\eta = 0.1$)

- $w_{11}^{(1)} = 2 - 0.1(0) = 2$
- $w_{12}^{(1)} = -1 - 0.1(-12) = -1 + 1.2 = 0.2$

Gradients for each weight: $\frac{\partial L}{\partial w} = \frac{\partial L}{\partial \hat{y}} \times \frac{\partial \hat{y}}{\partial h} \times \frac{\partial h}{\partial w}$

- $\frac{\partial L}{\partial w_{11}} = \frac{\partial L}{\partial \hat{y}} \times \frac{\partial \hat{y}}{\partial h_1} \times \frac{\partial h_1}{\partial w_{11}} = (-4) \times 2 \times 0 = 0$
- $\frac{\partial L}{\partial w_{12}} = \frac{\partial L}{\partial \hat{y}} \times \frac{\partial \hat{y}}{\partial h_2} \times \frac{\partial h_2}{\partial w_{12}} = (-4) \times (-1) \times 2 = 8$
- $\frac{\partial L}{\partial w_{21}} = \frac{\partial L}{\partial \hat{y}} \times \frac{\partial \hat{y}}{\partial h_1} \times \frac{\partial h_1}{\partial w_{21}} = (-4) \times 2 \times 0 = 0$
- $\frac{\partial L}{\partial w_{22}} = \frac{\partial L}{\partial \hat{y}} \times \frac{\partial \hat{y}}{\partial h_2} \times \frac{\partial h_2}{\partial w_{22}} = (-4) \times (-1) \times 5 = 20$

Update rule:

- $w_{11} = 2 - 0.1(0) = 2$
- $w_{21} = -3 - 0.1(0) = -3$
- $w_{12} = -1 - 0.1(8) = -1.8$
- $w_{22} = 1 - 0.1(20) = -1$

The Core Idea: Minimizing a Loss Function $J(\theta)$

We can find the minimum of a function either:

- **Analytically:** By solving $\nabla J(\theta) = 0$ for the parameters θ . (Exact, but often not possible).
- **Numerically:** By using an iterative algorithm like Gradient Descent. (Approximate, but very powerful).

The Gradient Descent Algorithm

The entire algorithm is captured by one fundamental update rule. We repeatedly take small steps in the direction of the negative gradient:

$$\theta_{\text{new}} = \theta_{\text{old}} - \eta \nabla J(\theta_{\text{old}})$$

Where η is the learning rate.

Thank you. Questions?